System Exploit BOF (Buffer Overflow)

1. Introduzione

L'obiettivo dell'esercizio è analizzare un programma scritto in linguaggio C, comprenderne il funzionamento e individuare una possibile vulnerabilità di tipo **Buffer Overflow (BOF)**. Successivamente, è richiesto di modificare il codice in modo da provocare un errore di segmentazione **(segmentation fault)** e infine implementare contromisure di sicurezza.

Il ***buffer overflow*** rappresenta una delle vulnerabilità più note nell'ambito della sicurezza informatica: si verifica quando un programma scrive dati oltre i limiti di memoria allocata, causando comportamenti imprevisti o potenzialmente sfruttabili.

2. Analisi del programma originale

Il programma fornito ha la seguente struttura logica:

2.1 Descrizione generale

- Definisce un array di 10 interi (`vector[10]`)
- Richiede all'utente di inserire 10 numeri
- Stampa il vettore inserito
- Ordina il vettore usando l'algoritmo **Bubble Sort**
- Stampa il vettore ordinato

2.2 Analisi dettagliata del codice

Dichiarazione delle variabili

```
int vector[10], i, j, k;
int swap_var;
```

- `vector[10]`: array di dimensione fissa
- `i, j, k`: contatori per i cicli
- `swap_var`: variabile temporanea per lo scambio

Input dell'utente

```
for (i = 0; i < 10; i++) {
   scanf("%d", &vector[i]);
}
```

- Il programma legge esattamente 10 interi
- Non sono presenti controlli sull'input

Stampa del vettore

```
for (i = 0; i < 10; i++) {
   printf("%d", vector[i]);
}
```

Ordinamento (Bubble Sort)

Il programma utilizza il Bubble Sort:

```
for (j = 0; j < 10 - 1; j++) {
   for (k = 0; k < 10 - j - 1; k++) {
       if (vector[k] > vector[k+1]) {
           swap_var = vector[k];
           vector[k] = vector[k+1];
           vector[k+1] = swap_var;
       }
   }
}
```

Funzionamento:

- Confronta coppie adiacenti
- Scambia i valori se non sono in ordine
- Ripete il processo fino a ordinamento completo

3. Esecuzione del programma

3.1 Compilazione

```
gcc programma.c -o programma
```

3.2 Esecuzione

```
./programma
```

3.3 Output atteso

- Inserimento di 10 numeri
- Visualizzazione del vettore originale
- Visualizzazione del vettore ordinato

4. Analisi della vulnerabilità

4.1 Problema principale

Il programma **non valida l'input dell'utente**.

In particolare:

- Non verifica il tipo di dato
- Non verifica il numero di input inseriti
- Non protegge la memoria da accessi fuori limite

4.2 Concetto di Buffer Overflow

Un buffer overflow si verifica quando:

- Si scrivono più dati di quelli che il buffer può contenere
- Si sovrascrive memoria adiacente

Nel nostro caso:

```
int vector[10];
```

può contenere **solo 10 interi**.

5. Modifica per generare un Buffer Overflow

Per provocare un errore di segmentazione, si può modificare il ciclo di input:

```
for (i = 0; i < 20; i++) {
   scanf("%d", &vector[i]);
}
```

5.1 Cosa succede

- Il programma tenta di scrivere oltre i limiti dell'array
- Si accede a memoria non allocata
- Possibile:
    - crash del programma
    - segmentation fault

5.2 Spiegazione tecnica

In memoria:

```
[ vector[0] ... vector[9] ][ altre variabili ]
```

Scrivendo oltre:

```
vector[10], vector[11], ...
```

→ si sovrascrivono aree di memoria non autorizzate.

6. Segmentation Fault

Un segmentation fault è un errore generato quando:

- il programma accede a memoria non consentita

Nel nostro caso:

- causato da scrittura fuori dai limiti dell'array

Esempio output:

```
Segmentation fault (core dumped)
```

7. Implementazione delle contromisure

7.1 Controllo dei limiti

```
for (i = 0; i < 10; i++) {
   if (scanf("%d", &vector[i]) != 1) {
       printf("Errore input\n");
       return 1;
   }
}
```

7.2 Validazione input

- Verifica che l'input sia numerico
- Evita valori non validi

7.3 Uso di costanti

```
#define SIZE 10
int vector[SIZE];
```

8. Bonus – Menu utente

Implementazione di un menu:

```
int scelta;
printf("1. Modalità sicura\n");
printf("2. Modalità vulnerabile\n");
scanf("%d", &scelta);
```

8.1 Modalità sicura

- Input limitato a 10 elementi
- Controlli attivi

8.2 Modalità vulnerabile

- Permette overflow
- Dimostra la vulnerabilità

9. Struttura della memoria (Stack) e Buffer Overflow

Per comprendere il Buffer Overflow è fondamentale analizzare come le variabili vengono allocate nello **stack**.

9.1 Memoria prima dell'overflow

Durante l'esecuzione della funzione `main`, le variabili locali vengono salvate nello stack:

```
+-----------------------------+
|       Return Address        |
+-----------------------------+
|     Variabili locali        |
|-------------------------------|
| swap_var                    |
| k                           |
| j                           |
| i                           |
| vector[9]                   |
| vector[8]                   |
| vector[7]                   |
| vector[6]                   |
| vector[5]                   |
| vector[4]                   |
| vector[3]                   |
| vector[2]                   |
| vector[1]                   |
| vector[0]                   |
+-----------------------------+
```

- Lo stack cresce verso il basso
- `vector` occupa uno spazio contiguo di memoria
- Le variabili successive sono salvate subito dopo

9.2 Memoria durante overflow

Se il programma scrive oltre `vector[9]`:

---

vector[10] → sovrascrive i
vector[11] → sovrascrive j
vector[12] → sovrascrive k
vector[13] → sovrascrive swap_var

Diagramma:

```
+----------------------------------------+
|      Return Address                    | ← possibile target critico
+----------------------------------------+
| swap_var   ← CORROTTO                  |
| k          ← CORROTTO                  |
| j          ← CORROTTO                  |
| i          ← CORROTTO                  |
| vector[9]                              |
| vector[8]                              |
| vector[7]                              |
| vector[6]                              |
| vector[5]                              |
| vector[4]                              |
| vector[3]                              |
| vector[2]                              |
| vector[1]                              |
| vector[0]                              |
+----------------------------------------+
```

9.3 Implicazioni di sicurezza

Un attaccante può:

- corrompere variabili
- alterare il flusso del programma
- nei casi peggiori → sovrascrivere il **return address**

Questo è il principio base degli exploit reali.

10. Codice completo – Versione Vulnerabile

Questa versione contiene volutamente un errore che può causare Buffer Overflow.

BW_D3_BOF 1.c

```c
// Versione Vulnerabile

#include <stdio.h>

int main() {

    int vector[10], i, j, k;
    int swap_var;

    printf("MODALITA' VULNERABILE\n");
    printf("Inserire 20 interi:\n");

    // ERRORE: supera la dimensione del buffer
    for (i = 0; i < 20; i++) {
        printf("[%d]: ", i+1);
        scanf("%d", &vector[i]); // BUFFER OVERFLOW
    }

    printf("\nVettore inserito:\n");
    for (i = 0; i < 10; i++) {
        printf("%d ", vector[i]);
    }

    // Bubble sort
    for (j = 0; j < 10 - 1; j++) {
        for (k = 0; k < 10 - j - 1; k++) {
            if (vector[k] > vector[k+1]) {
                swap_var = vector[k];
                vector[k] = vector[k+1];
                vector[k+1] = swap_var;
            }
        }
    }

    printf("\nVettore ordinato:\n");
    for (j = 0; j < 10; j++) {
        printf("%d ", vector[j]);
    }

    return 0;
}
```

11. Codice completo – Versione Sicura

Questa versione corregge la vulnerabilità.

BW_D3_BOF 2.c

```c
// Versione Sicura

#include <stdio.h>
#define SIZE 10

int main() {

    int vector[SIZE], i, j, k;
    int swap_var;

    printf("MODALITA' SICURA\n");
    printf("Inserire 10 interi:\n");

    for (i = 0; i < SIZE; i++) {
        printf("[%d]: ", i+1);

        if (scanf("%d", &vector[i]) != 1) {
            printf("Errore: input non valido\n");
            return 1;
        }
    }

    printf("\nVettore inserito:\n");
    for (i = 0; i < SIZE; i++) {
        printf("%d ", vector[i]);
    }

    // Bubble sort
    for (j = 0; j < SIZE - 1; j++) {
        for (k = 0; k < SIZE - j - 1; k++) {
            if (vector[k] > vector[k+1]) {
                swap_var = vector[k];
                vector[k] = vector[k+1];
                vector[k+1] = swap_var;
            }
        }
    }

    printf("\nVettore ordinato:\n");
    for (j = 0; j < SIZE; j++) {
        printf("%d ", vector[j]);
    }

    return 0;
}
```

12. Codice completo – Versione con Menu (Bonus)

Questa versione integra sia modalità sicura che vulnerabile.

BW_D3_BOF 3.c

```c
// Versione con Menu (Bonus)

#include <stdio.h>
#define SIZE 10

void bubble_sort(int vector[]) {
    int j, k, swap_var;

    for (j = 0; j < SIZE - 1; j++) {
        for (k = 0; k < SIZE - j - 1; k++) {
            if (vector[k] > vector[k+1]) {
                swap_var = vector[k];
                vector[k] = vector[k+1];
                vector[k+1] = swap_var;
            }
        }
    }
}

int main() {

    int vector[SIZE], i;
    int scelta;

    printf("1. Modalita' Sicura\n");
    printf("2. Modalita' Vulnerabile\n");
    printf("Scelta: ");
    scanf("%d", &scelta);

    if (scelta == 1) {

        printf("\n[MODALITA' SICURA]\n");

        for (i = 0; i < SIZE; i++) {
            printf("[%d]: ", i+1);

            if (scanf("%d", &vector[i]) != 1) {
                printf("Errore input\n");
                return 1;
            }
        }

    } else if (scelta == 2) {

        printf("\n[MODALITA' VULNERABILE]\n");

        for (i = 0; i < 20; i++) { // volutamente errato
            printf("[%d]: ", i+1);
            scanf("%d", &vector[i]); // overflow
        }

    } else {
        printf("Scelta non valida\n");
        return 1;
    }

    printf("\nVettore inserito:\n");
    for (i = 0; i < SIZE; i++) {
        printf("%d ", vector[i]);
    }

    bubble_sort(vector);

    printf("\nVettore ordinato:\n");
    for (i = 0; i < SIZE; i++) {
        printf("%d ", vector[i]);
    }

    return 0;
}
```

13. Considerazioni finali

L'esercizio ha permesso di:

- comprendere il funzionamento di un programma C
- analizzare un algoritmo di ordinamento (Bubble Sort)
- identificare una vulnerabilità reale
- simulare un attacco di tipo Buffer Overflow
- implementare contromisure

Il **Buffer Overflow** rappresenta una minaccia critica nella sicurezza software. La prevenzione richiede:

- validazione rigorosa dell'input
- gestione sicura della memoria
- utilizzo di buone pratiche di programmazione

Questo laboratorio dimostra come anche programmi semplici possano contenere vulnerabilità gravi. In ambito cybersecurity, è fondamentale:

- analizzare il codice
- testare comportamenti anomali
- prevenire exploit

14. Conclusione

Questo esercizio dimostra in modo pratico come vulnerabilità apparentemente semplici possano avere impatti critici sulla sicurezza di un sistema. **Il Buffer Overflow**, in particolare, rappresenta una delle tecniche più sfruttate negli attacchi informatici, specialmente in contesti low-level come il linguaggio C.

L'analisi dello stack evidenzia come la mancanza di controlli sull'input possa portare alla corruzione della memoria e, nei casi più avanzati, al controllo del flusso di esecuzione del programma.

L'adozione di pratiche sicure, come:

- validazione dell'input
- uso di costanti
- limitazione degli accessi alla memoria

è fondamentale per sviluppare software robusti e resistenti agli attacchi.